Colombo Metro Delivery Route Optimisation

Exact (Held-Karp) and heuristic solutions to a 100-node Travelling Salesman Problem

Optimization Methods – Programming Assignment (Group of 2)

October 2026

Group members: [FULL NAME AND STUDENT ID - fill in members.txt] · [FULL NAME AND STUDENT ID - fill in members.txt]

Code, data and commit history: [GITHUB URL - fill in submission.txt]

1. Problem selection and context

A courier based at Colombo Fort must deliver parcels to 99 localities spread over the Colombo, Gampaha and Kalutara districts, from Kochchikade in the north to Kalutara in the south and Avissawella in the east. One vehicle starts at the depot, visits every locality exactly once and returns. The objective is to minimise the total distance driven, which is the classic **Travelling Salesperson Problem (TSP)**.

The problem is realistic because every courier, field-service and waste-collection round has this structure, and it is non-trivial because it is NP-hard: with n stops there are $(n-1)!/2$ distinct tours (about 10^{155} for $n = 100$), so enumeration is impossible and the quality of the solution method matters. It also lets us study the trade-off the assignment asks about: an exact method that gives a proof of optimality but scales exponentially, against heuristics that scale well but give no guarantee. Real deployments would add capacities and time windows; Section 9 explains how the model extends to those.

2. Data description

The instance has **100 nodes**: node 0 is the depot (Colombo Fort) and nodes 1–99 are named localities. The file `data/colombo_delivery_locations.csv` stores node id, name, latitude and longitude. The distance between two nodes is the great-circle (Haversine) distance in kilometres, so the cost matrix is symmetric and satisfies the triangle inequality. `data/distance_matrix_km.csv` is the derived matrix; the solvers recompute it from the locations file and a test checks that the two agree.

Provenance and quality checks. The locality names are real places, but the coordinates are approximate representative centre points curated by the team, not customer addresses. Because real-world data is preferred, every name was cross-checked against OpenStreetMap (Nominatim, retrieved 14 October 2026; script `src/validate_against_osm.py`, output `data/osm_validation.csv`). Of the 102 names in the first version of the file, 95 returned a match; the median offset between our point and OSM was 0.75 km and 77 of the 95 were within 2 km. The check found real errors, which were fixed:

- Bandaragama was 21.7 km from its OSM position and Ganemulla 5.2 km; both were corrected.
- Pannipitiya Junction and Moratuwa Town had *exactly* the same coordinates as Pannipitiya and Moratuwa, adding zero-distance nodes, so they were removed (102 → 100 nodes).

- "Kochchikade" was renamed "Kochchikade (Negombo)" to avoid confusion with a Colombo-15 locality of the same name.

Several other OSM matches were homonyms or district centroids and were reviewed by hand and left unchanged (column `status` in the validation file). Some localities are very close (e.g. Narahenpita and Thimbirigasyaya, 0.11 km apart); they are distinct places but make part of the instance easy. **Limitations:** straight-line distances are shorter than road distances, and the instance has no demands or time windows. The data is used only to instantiate and evaluate the optimisation models, not for training.

3. Mathematical formulation

Let $N = \{0, 1, \dots, n-1\}$ be the nodes with depot 0, $C = N \setminus \{0\}$ the customers and c_{ij} the Haversine distance from i to j. Decision variables:

- $x_{ij} \in \{0, 1\}$ for $i \neq j$: equal to 1 if the vehicle drives directly from i to j;
- $u_i \in \mathbb{R}$ for $i \in C$: the position of customer i in the tour (used only to forbid sub-tours).

$$\min \sum_{i \in N} \sum_{j \in N,\, j \neq i} c_{ij}\, x_{ij}$$

subject to

$$\sum_{j \in N,\, j \neq i} x_{ij} = 1 \quad \forall i \in N \qquad \text{(leave every node exactly once)}$$

$$\sum_{i \in N,\, i \neq j} x_{ij} = 1 \quad \forall j \in N \qquad \text{(enter every node exactly once)}$$

$$u_i - u_j + (n-1)\, x_{ij} \leq n - 2 \quad \forall i, j \in C,\ i \neq j \qquad \text{(Miller–Tucker–Zemlin sub-tour elimination)}$$

$$1 \leq u_i \leq n-1 \quad \forall i \in C, \qquad x_{ij} \in \{0, 1\}.$$

The degree constraints alone allow several disjoint cycles; the MTZ constraints force the order variables u to increase along any route that does not pass through the depot, which rules out a cycle on customers only, so every feasible x is one Hamiltonian cycle through the depot. An equivalent formulation replaces the MTZ rows by the Dantzig–Fulkerson–Johnson constraints $\sum_{i \in S} \sum_{j \in S,\, j \neq i} x_{ij} \leq |S| - 1$ for every $S \subseteq C$, $|S| \geq 2$, which is tighter but has exponentially many rows. **Assumptions:** one vehicle, symmetric costs, every locality visited once, no capacity, service time or time window.

4. Exact method: Held–Karp dynamic programming

Why this method. The assignment allows ILP, branch and bound or dynamic programming. Held–Karp needs no solver licence, is easy to verify (we check it against brute-force enumeration on random 8-node instances), and its runtime and memory are predictable, which suits a scalability study.

Recurrence. Let $C(S, j)$ be the length of the shortest path that starts at the depot, visits exactly the customers in $S \subseteq C$ and ends at $j \in S$:

$$C(\{j\}, j) = c_{0j}, \qquad C(S, j) = \min_{k \in S \setminus \{j\}} \big[C(S \setminus \{j\}, k) + c_{kj}\big],$$

$$\text{optimal tour length} = \min_{j \in C} [C(C, j) + c_{j0}].$$

Parent pointers recover the tour. There are $(n-1)2^{n-1}$ states with $O(n)$ work each, so time is $O(n^2 2^n)$ and memory $O(n2^n)$. Our array-based implementation is in `src/tsp/exact.py`. Because the DP table for $n = 30$ would already need about 1.6×10^{10} entries, the exact method is applied to random subsets of up to 20 nodes (Section 6), and for the full 100 nodes ($\approx 6 \times 10^{31}$ entries) it is impossible.

For the full instance we also compute the **1-tree lower bound** (minimum spanning tree on the customers plus the two cheapest depot edges; Held and Karp, 1970). Every tour contains such a structure, so no tour can be shorter. The bound is cheap but weak, and is used only to put a provable limit on the optimality gap.

5. Heuristic methods

All heuristics are in `src/tsp/heuristics.py`. A tour is a list of node ids that starts and ends at the depot.

1. **Nearest Neighbour (NN)** – start at the depot and repeatedly drive to the closest unvisited locality. $O(n^2)$, very fast, but greedy: early choices leave long return trips at the end.
2. **2-opt** – repeatedly remove two edges and reconnect the tour by reversing the segment between them whenever this shortens the tour (Croes, 1958). It removes crossing edges. Moves are evaluated from four distances only, so one pass costs $O(n^2)$.
3. **2-opt + Or-opt** – additionally move a segment of 1–3 consecutive customers (also reversed) to a cheaper place in the tour (Or, 1976). The two neighbourhoods are alternated until neither improves the tour.
4. **Multi-start 2-opt** – run NN from 30 different random first customers, improve each with 2-opt and keep the best (diversification).
5. **Iterated local search (ILS)** – start from the local optimum of method 3, then repeat 300 times: apply a random *double-bridge* perturbation (cut the tour into three pieces and reorder them), re-optimise with 2-opt + Or-opt and keep the result only if the tour became shorter (Lourenço et al., 2003). This lets the search escape local optima that 2-opt and Or-opt cannot leave.

Why these methods. NN is the standard baseline that every improvement is measured against. 2-opt and Or-opt are the best-known simple local searches for the TSP, and they are complementary: 2-opt repairs crossings, Or-opt repairs badly placed single stops. Multi-start and ILS are two ways of spending extra time to escape local optima, so the comparison also shows what additional runtime buys. Randomised methods use fixed seeds, so every number in this report can be reproduced.

6. Experimental design

Study A – quality against the proven optimum (scalability study). For each size $n \in \{8, 10, 12, 14, 16, 18, 20\}$ (depot included) we draw 5 random subsets of the 100 localities (fixed seeds), solve each with Held–Karp, and run all heuristics on the same subset (to keep these many runs cheap, multi-start uses 10 starts and ILS 50 iterations here). We record the optimal length, the runtime and the percentage gap of each heuristic to the optimum.

Study B – full instance. All five heuristics run on the complete 100-node instance. Deterministic methods are timed over 5 repeats; randomised methods use 5 different seeds. We report best, mean and standard deviation of the tour length, the median runtime, the gap to the best tour found, and

the gap to the 1-tree lower bound. Every returned tour is checked for **feasibility** (starts and ends at the depot, every node exactly once).

Code runs in pure Python 3 (standard library) on one machine; absolute runtimes will differ on another computer, but the ratios are meaningful. Everything is reproduced by `python src/run_experiments.py`.

7. Results

7.1 Heuristics against the exact optimum

Table 1 gives the mean gap to the optimum over 5 subsets per size. Figure 1 shows runtimes and Figure 2 the gaps.

Table 1: Mean gap (%) to the Held–Karp optimum; exact runtime is the median over 5 subsets.

n	Exact (s)	NN	2-opt	2-opt+Or	Multi-start	ILS
8	0.00032	11.04	0.06	0.00	0.06	0.00
10	0.00207	12.28	1.68	0.00	0.13	0.00
12	0.01054	11.15	0.69	0.20	0.00	0.00
14	0.05994	24.49	1.27	0.91	0.00	0.00
16	0.3093	21.13	3.43	0.00	0.19	0.00
18	1.556	14.17	2.47	0.61	0.19	0.00
20	8.051	18.34	3.87	0.51	0.31	0.00

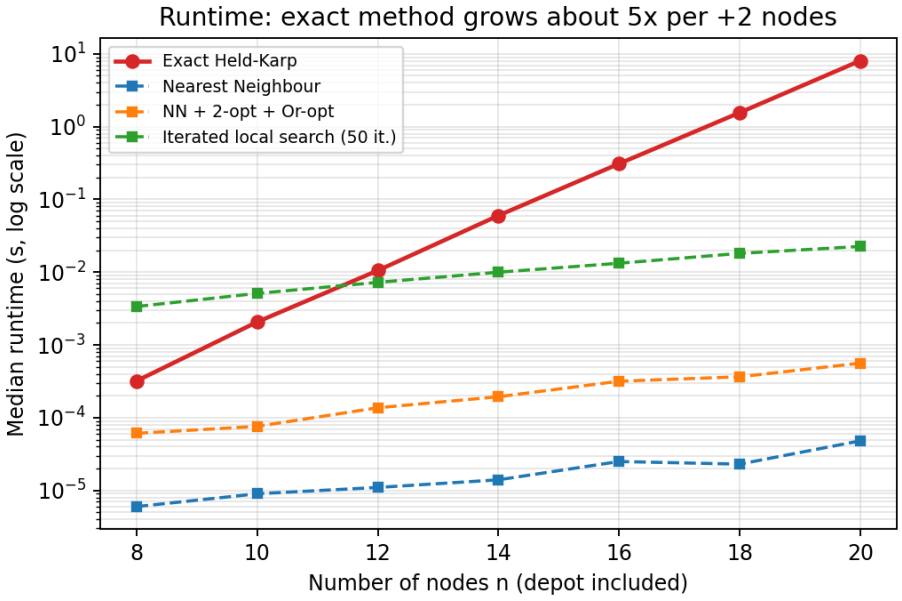


Figure 1: Median runtime against instance size (log scale).

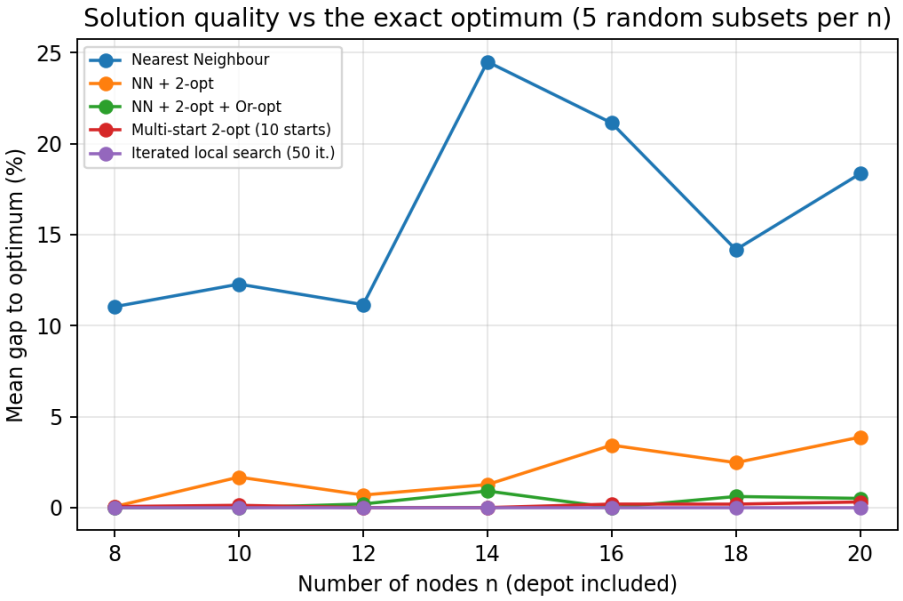


Figure 2: Mean gap to the optimum against instance size.

7.2 Full 100-node instance

Table 2: Results on the full instance (5 runs per method). Gap columns refer to the best tour of each method.

Method	Best (km)	Mean (km)	Std (km)	Median time (s)	Gap to best found (%)	Gap to lower bound (%)
Nearest Neighbour	432.7243	432.72	0.00	0.0002	29.19	60.60
NN + 2-opt	355.0853	355.09	0.00	0.00256	6.01	31.79
NN + 2-opt + Or-opt	342.0867	342.09	0.00	0.01818	2.13	26.96
Multi-start 2-opt (30 starts)	338.6396	338.73	0.11	0.07924	1.10	25.68
Iterated local search (300 it.)	334.9597	334.96	0.00	4.542	0.00	24.32

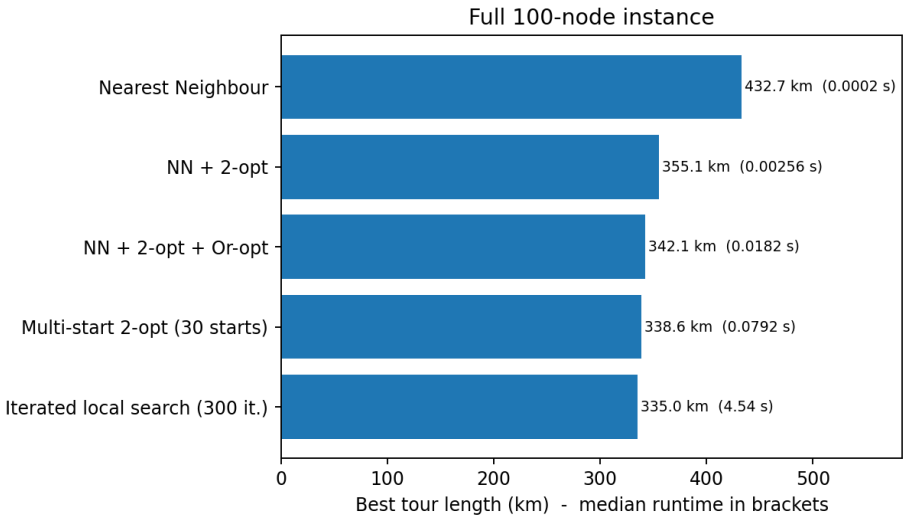


Figure 3: Best tour length and median runtime per method.

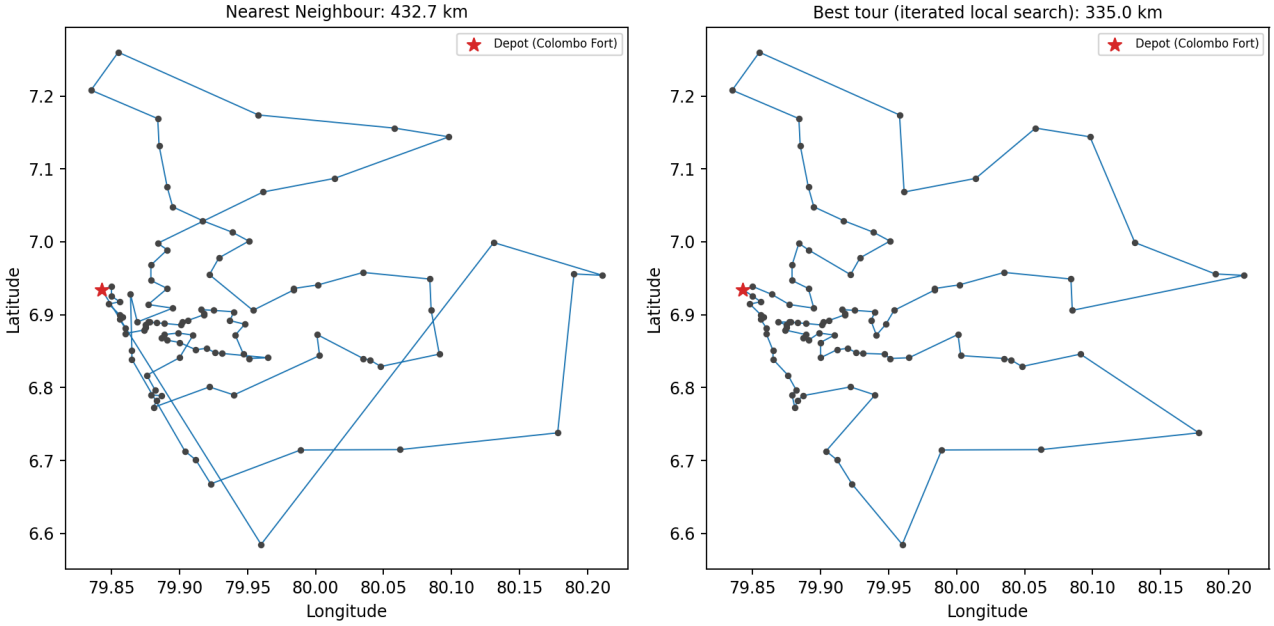


Figure 4: Nearest Neighbour tour (left) and best tour found (right).

8. Discussion

Solution quality. Nearest Neighbour is 11–24% above the optimum on the small instances and 29.2% above the best known tour on the full instance; its route in Figure 4 (left) has long crossing legs back to the depot area. 2-opt cuts the gap to 0.1–3.9% on the small instances and the full-instance length from 432.7 km to 355.1 km. Adding Or-opt reaches 342.1 km. Multi-start gives 338.6 km and ILS gives 335.0 km. **ILS found the proven optimum on all 35 small instances** (gap 0.00% for every size), and on the full instance all five seeds ended on the same tour length, which suggests it is very close to

optimal, although for 100 nodes this is evidence, not proof.

What the lower bound says. The 1-tree bound is 269.4 km, so the best tour is at most 24.3% above the optimum. This bound is known to be loose; we state it because it is the only *provable* statement available at this size, and we do not claim that the real gap is anywhere near 24.3%. The small-instance study is the better indicator of heuristic quality.

Runtime and scalability. The exact method multiplies its runtime by about 5 for every two extra nodes (0.3 ms at $n = 8$, 8 s at $n = 20$). Extrapolating, $n = 30$ would take roughly eight hours (extrapolated from our measurements) and the DP table would not fit in memory; $n = 100$ is out of reach by many orders of magnitude. The heuristics grow gently: at $n = 100$ NN takes 0.0002 s, 2-opt + Or-opt 0.018 and ILS 4.5 s. ILS is about 250 times slower than 2-opt + Or-opt but buys a 2.1% shorter route. For a daily plan that is a good trade; for a real-time re-route the faster local search may be preferred.

Feasibility. All tours were verified feasible. The exact method guarantees optimality only up to 20 nodes; for the heuristics feasibility is guaranteed by construction and re-checked in every run.

Trade-offs in one sentence. The exact method gives certainty but stops being usable at about 25 nodes; 2-opt + Or-opt is nearly free and typically within a few percent; ILS spends seconds to reach what appears to be the optimum.

9. Limitations and future work

- **Distances.** Haversine distance underestimates road distance (and the ranking of routes may change). Replace it by road-network distances from OSRM, GraphHopper or OSMnx and keep a dated OSM snapshot.
- **Data.** Coordinates are approximate locality centres; OSM validation reduced errors but a real deployment needs geocoded customer addresses.
- **Model.** One vehicle, no demand, capacity, service time, time windows or traffic. The natural extensions are the capacitated VRP and VRP with time windows; the heuristics extend with route-splitting and Or-opt/relocate moves between routes, and the ILP gains vehicle-load and time variables.
- **Exact method.** An ILP with sub-tour cuts (branch-and-cut, e.g. via PuLP/OR-Tools or Concorde) would give the true optimum or a much tighter bound for 100 nodes; a Held–Karp subgradient bound would replace our weak 1-tree bound.
- **Experiments.** Runtimes come from one machine and one language; ILS parameters (300 iterations, double-bridge) were not tuned; a stronger metaheuristic (Lin–Kernighan, simulated annealing, genetic algorithm) could be compared.

10. Individual contributions

Area	Member 1 (M1)	Member 2 (M2)
Problem and data	Dataset curation, OSM cross-check, correction of errors and duplicates (`src/validate_against_osm.py`, `src/tsp/data.py`, `data/`)	Review of the cleaned data and its limitations

Area	Member 1 (M1)	Member 2 (M2)
Exact method	Held–Karp, 1-tree bound, brute-force verification (`src/tsp/exact.py`)	Review and testing of the exact solver
Heuristics	–	NN, 2-opt, Or-opt, multi-start, ILS (`src/tsp/heuristics.py`) and unit tests
Experiments and figures	Scalability study design	Experiment runner, repeated runs, plots (`src/run_experiments.py`, `src/make_plots.py`)
Report	Sections 1–4 (problem, data, formulation, exact method)	Sections 5–9 (heuristics, experiments, results, discussion, future work)
Video	Parts on problem, data, formulation and exact method	Parts on heuristics, results, discussion and conclusion

The commit history in the repository records the work: commit messages are prefixed `M1:` or `M2:`. Both members can explain the whole code base.

References

1. Dantzig, G., Fulkerson, R., Johnson, S. (1954). Solution of a large-scale traveling-salesman problem. *Operations Research* 2(4), 393–410.
2. Miller, C. E., Tucker, A. W., Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. *Journal of the ACM* 7(4), 326–329.
3. Held, M., Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the SIAM* 10(1), 196–210.
4. Held, M., Karp, R. M. (1970). The traveling-salesman problem and minimum spanning trees. *Operations Research* 18(6), 1138–1162.
5. Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research* 6(6), 791–812.
6. Or, I. (1976). *Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking*. PhD thesis, Northwestern University.
7. Lourenço, H. R., Martin, O. C., Stützle, T. (2003). Iterated local search. In *Handbook of Metaheuristics*, Springer, 320–353.
8. OpenStreetMap contributors. OpenStreetMap data, © OpenStreetMap contributors, ODbL 1.0. https://www.openstreetmap.org/copyright (Nominatim geocoder, retrieved 14 October 2026).